Experiential Learning
Project Assessment-I
on
"Rainfall Data Analyzer"
under the course
Python Programming Language (N-PCCCM304P)

Presented By

Swanand S. Barve

Under the Guidance of

Mrs. Mayuri Getme

Session:2026-27 (ODD)

Department of CSE(AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH,NAGPUR

An Autonomous Institute, Affiliated to RTMNU, Nagpur

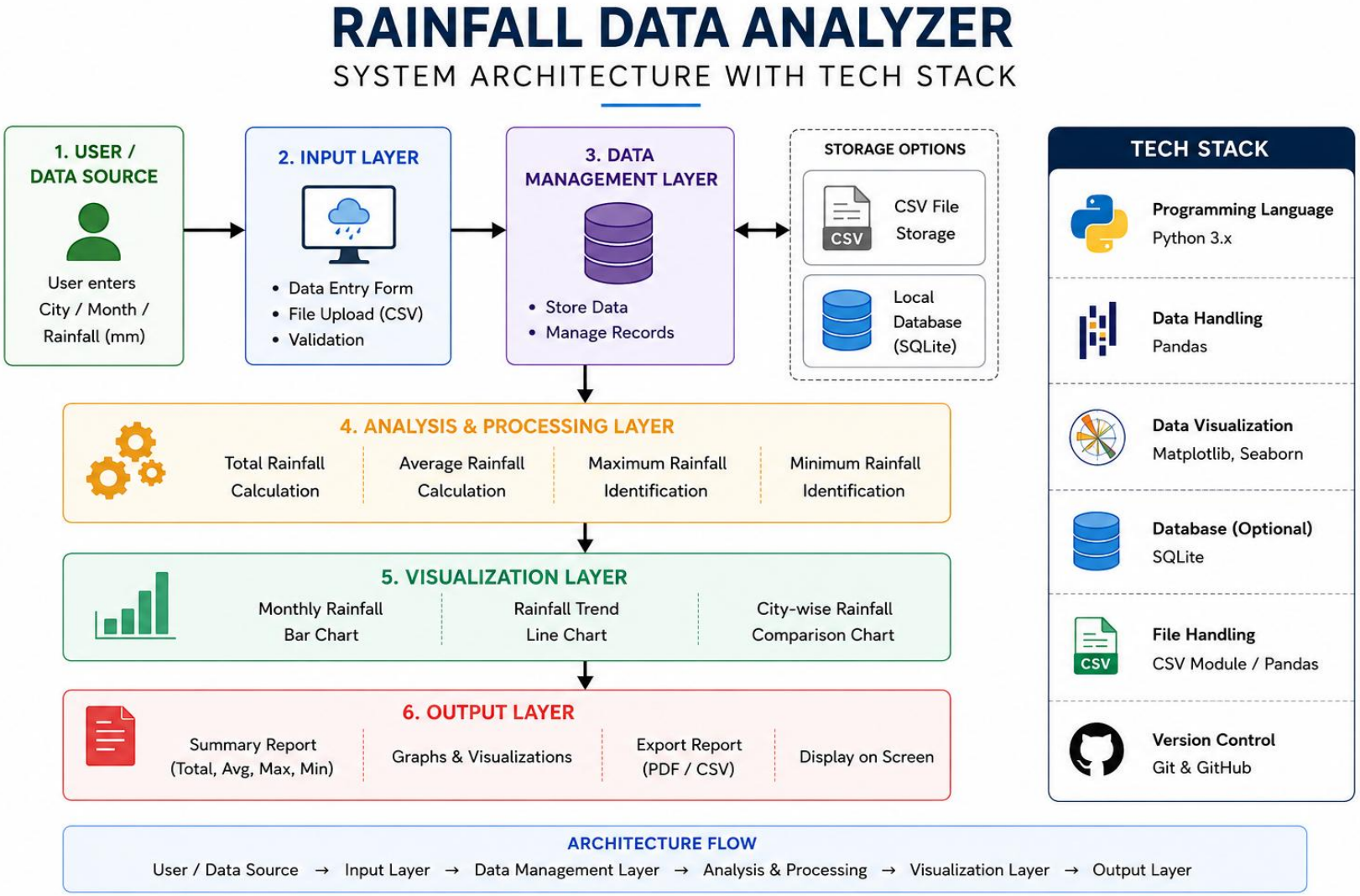
Problem Statement

Develop a Python application to record and analyze rainfall data for different cities or months. The system should calculate total rainfall, average rainfall, maximum and minimum rainfall, and generate summary reports and graphical visualizations.

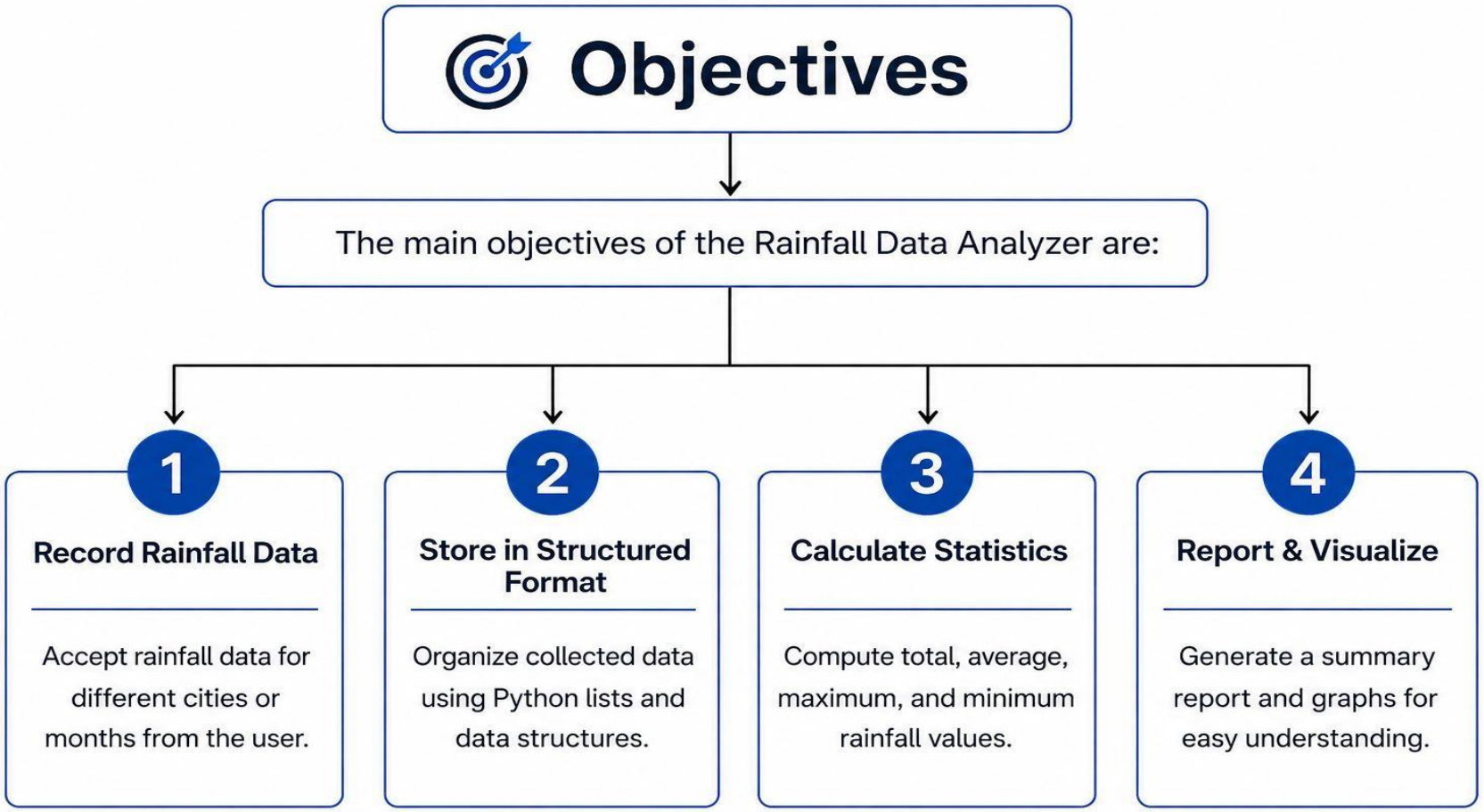
- **Total Rainfall**
  Sum of all recorded rainfall values across the dataset
- **Average Rainfall**
  Mean value to understand typical rainfall patterns
- **Maximum Rainfall**
  Identify the highest recorded rainfall event
- **Minimum Rainfall**
  Identify the lowest recorded rainfall event

The system should also generate a summary report and graphical representation for easy interpretation

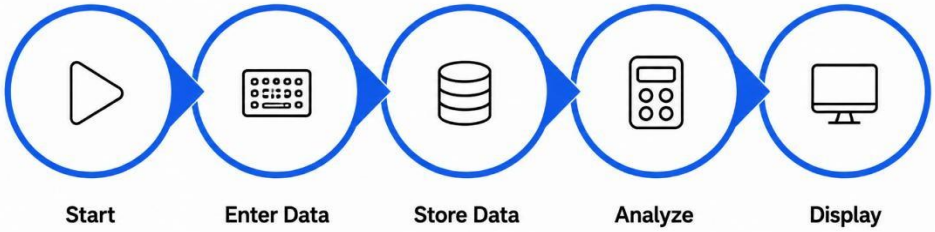
System Architecture & TechStack



Objectives


Objectives
The main objectives of the Rainfall Data Analyzer are:
1
Record Rainfall Data
Accept rainfall data for different cities or months from the user.
2
Store in Structured Format
Organize collected data using Python lists and data structures.
3
Calculate Statistics
Compute total, average, maximum, and minimum rainfall values.
4
Report & Visualize
Generate a summary report and graphs for easy understanding.

Algorithm & Pseudocode



01 Start the program

02 Enter Data

Input city/month and corresponding rainfall values.

03 Store Data

Save rainfall values in a list or suitable data structure

04 Analyze Data

Calculate total, average, maximum, and minimum minimnum rainfall

05 Display Results

Show all calculated statistics to the user

06 Generate Graph & Report

Create graphical visualization and summary report, report, then stop

Pseudocode

```
START
Input rainfall values

total = sum(rainfall)
average = total / number of values
maximum = max(rainfall)
minimum = min(rainfall)

Display total, average,
 maximum, minimum

Generate rainfall graph
END
```

Understanding of Python Logic

Python Concept	Purpose
Variables	Store rainfall values and results
Lists	Store multiple rainfall values
Input	Accept rainfall data from user
Loops	Process multiple entries
Functions	Organize program logic
sum()	Calculate total rainfall
max() / min()	Find maximum / minimum rainfall
Matplotlib	Create graphs and visualizations

Phase 1 vs Phase 2 – Project Progress

Feature	Phase 1	Phase 2
Problem identification	Problem identification	Working implementation
Basic Python logic	Basic Python logic	Interactive dashboard
Pseudocode	Pseudocode	Actual data processing
Basic graph	Basic graph	Multiple interactive charts
Manual rainfall input	Manual rainfall input	Form + CSV import
Basic calculations	Basic calculations	Automatic statistics
Python/Matplotlib	Python/Matplotlib	HTML + CSS + JavaScript + Chart.js

Application Overview – Pluvia Analytics

Pluvia Analytics is the Phase 2 implementation of the Rainfall Data Analyzer: a cross-platform **Flet** application that explores monthly rainfall trends across major Indian cities through a dark-mode, glassmorphism dashboard.

❖ At a glance

6	12	72	1991–2020
Indian cities	Months each	Rainfall records	ERA5 monthly means

Pandas data engine + analytics engine: total, average, maximum, minimum

Interactive charts (Matplotlib), next-month forecast, live weather from Open-Meteo

Record management, CSV import, and CSV / PDF / TXT export · **~2,700 lines** in a single `app.py`

Key Features

1

Record Management

Add records via dialog with validation (non-negative numbers), update duplicates, delete rows. Paginated table, 10 rows per page.

2

CSV Import

Upload City, Month, Rainfall (mm). Accepts short or full month names; all-or-nothing validation reports bad rows.

3

Dynamic Filters

City and Month dropdowns re-slice the data; KPIs, charts and table refresh instantly.

4

Live KPIs

Total, average monthly, peak and minimum rainfall with the city and month of each extreme.

5

Charts

City comparison bars, monthly trend line and a 7-day observed rainfall / temperature chart.

6

Forecast

Next-month estimate from a local linear trend blended with the seasonal norm, plus an alert badge.

7

Live Weather

Open-Meteo 7-day forecast cards, last-7-days chart and one-click sync of the latest month.

8

Export Reports

Download the filtered dataset as CSV, or the KPI summary as PDF or TXT.

Processing flow: Data → Filter → Analyze → Visualize → Report, refreshed on every change

Technology Stack Used

Component	Technology	Role in the application
Language	Python 3	Whole application – type hints, dataclasses, asyncio
UI Framework	Flet 1.0.4	Cross-platform responsive dashboard, dark glass theme
Data Handling	pandas	Long-format DataFrame, groupby, filtering, CSV parsing
Visualization	Matplotlib (Agg)	Charts rendered as transparent PNGs shown via ft.Image
Live Data	Open-Meteo APIs	Geocoding, historical archive and 7-day forecast (urllib)
Reports	CSV · PDF · TXT	Built-in dependency-free PDF writer for summaries
Version Control	Git & GitHub	Source hosted in the rainfall-data-analyzer repository

Install: `pip install flet pandas matplotlib` → Run: `flet run app.py`

Application Architecture (app.py)

1 Constants & Metadata

`RAINFALL_COL, MONTHS, CITIES and SAMPLE_RAINFALL_MM (1991–2020 monthly means)`

Fixed labels and the bundled 1991–2020 dataset

2 Sample Data Engine

`generate_sample_data()`
`parse_rainfall_csv()`
`to_wide_format()`

Builds, validates and reshapes the data

3 Core Analytics Engine

`compute_total / average · find_max / min · filter_by_city / month · compute_forecast`

Pure functions for every KPI and filter

4 Flet UI – Dark + Glass

`glass_container · KPI cards · filter bar · weather cards · compute_kpis()`

Dark glassmorphism components and KPI logic

5 Visualization Engine

`render_city_bar_chart · render_monthly_line_chart · render_past_week_chart`

Matplotlib charts in the app palette

6 main(page)

`Assembles dashboard; handles add, import, export, sync and filters`

Wires everything together and handles events

Flow: Data Engine → Analytics Engine → Visualization Engine → Flet UI → Export

Dataset & CSV Import

City	Total (mm)	Avg/mo	Wettest month
Mumbai	1,919.9	160.0	Jul (636.2)
Delhi	671.0	55.9	Jul (194.7)
Chennai	966.3	80.5	Nov (227.4)
Kolkata	1,596.7	133.1	Jul (348.3)
Bengaluru	905.2	75.4	Oct (138.4)
Nagpur	1,114.1	92.8	Jul (308.0)

Bundled data: 1991–2020 monthly means (Open-Meteo ERA5). Nagpur uses approximate climatological normals.

CSV import rules

- Required columns: `City, Month, Rainfall (mm)`
- Header aliases accepted (e.g. “rainfall”)
- Months: Jan or January
- Rainfall must be finite and ≥ 0
- **All-or-nothing:** any bad row rejects the file and the first 5 errors are shown
- Data kept in **long format** (one row per City–Month); wide format for tables

72
rows (long format)

3
columns: City · Month · Rainfall (mm)

6 × 12
City × Month matrix (wide)

Analytics Engine

Function	Purpose
`compute_total_rainfall`	Sum of all records
`compute_average_monthly_rainfall`	Mean monthly rainfall
`find_max / find_min_rainfall`	Wettest / driest city-month
`average_by_month`	Mean per month, Jan → Dec
`filter_by_city / month`	Case-insensitive slicing
`compute_kpis`	All four KPIs for a scope

```
def compute_total_rainfall(data):
    return float(data[RAINFALL_COL].sum())

def compute_average_monthly_rainfall(data):
    return float(data[RAINFALL_COL].mean())

def find_max_rainfall(data):
    i = data[RAINFALL_COL].idxmax()
    return RainfallExtreme(value, city, month)

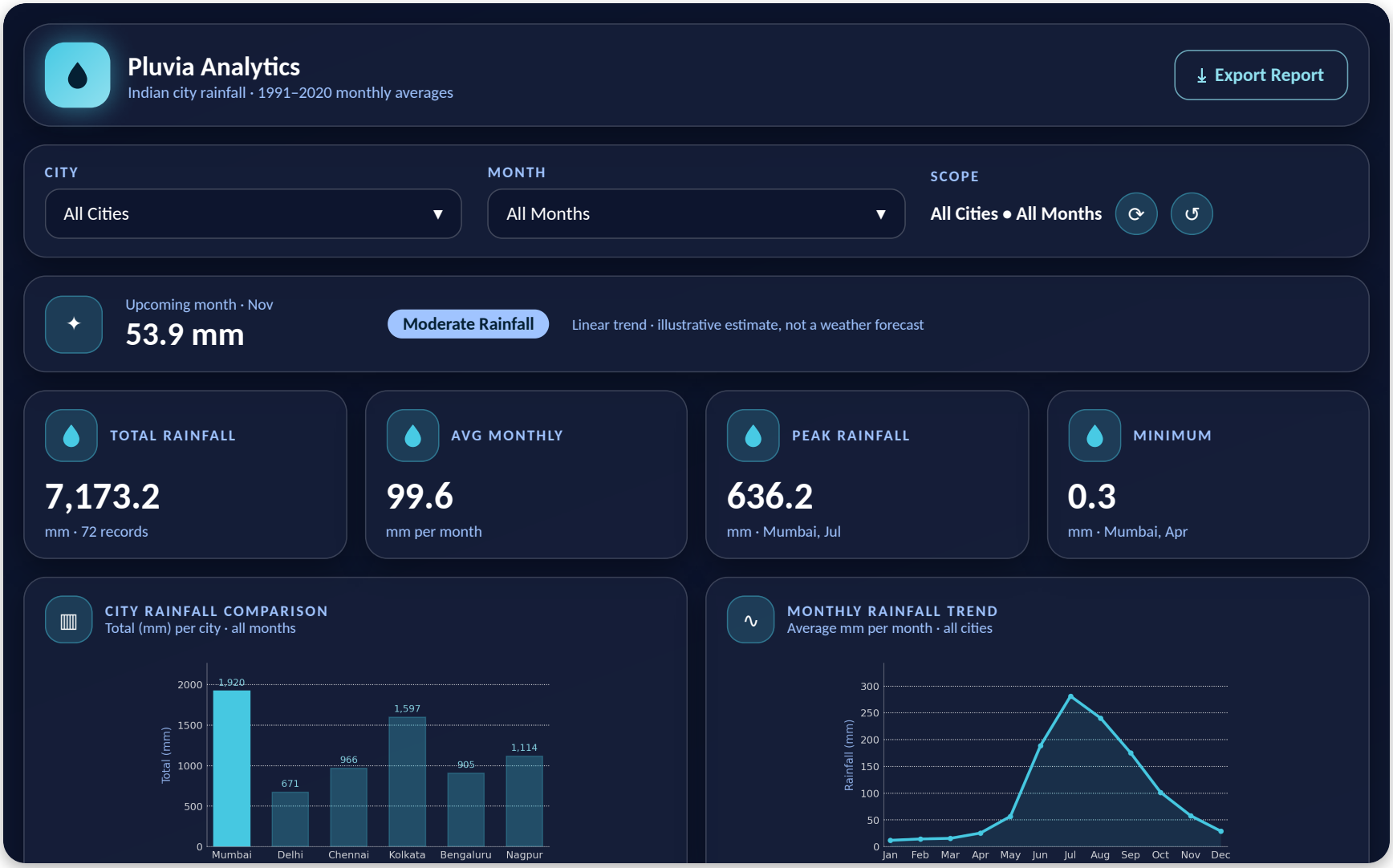
def find_min_rainfall(data):
    i = data[RAINFALL_COL].idxmin()
    return RainfallExtreme(value, city, month)
```

Output on the full dataset

Total rainfall	Average / month	Maximum	Minimum
7,173.2	**99.6**	**636.2**	**0.3**
mm · 72 records	mm	mm · Mumbai, Jul	mm · Mumbai, Apr

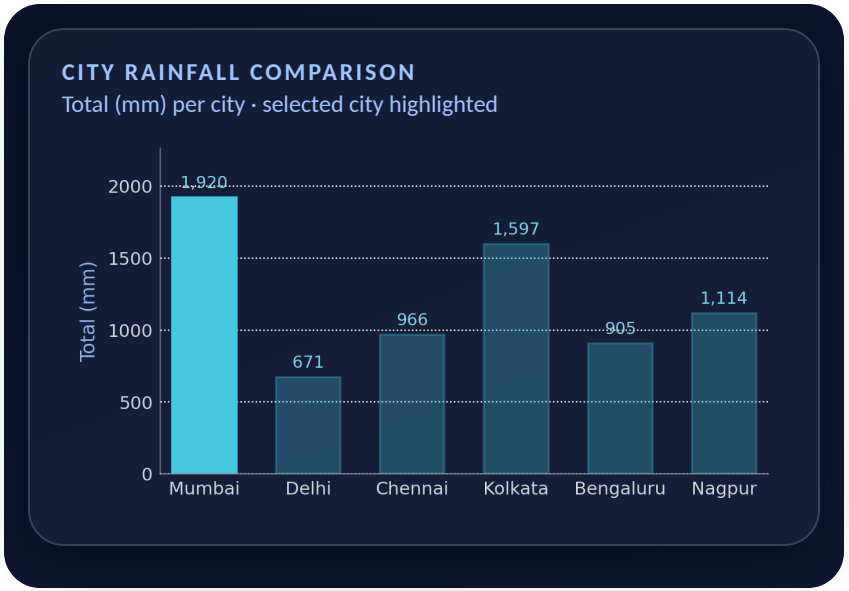
Every function takes a **pandas DataFrame**, so filters and metrics compose freely.

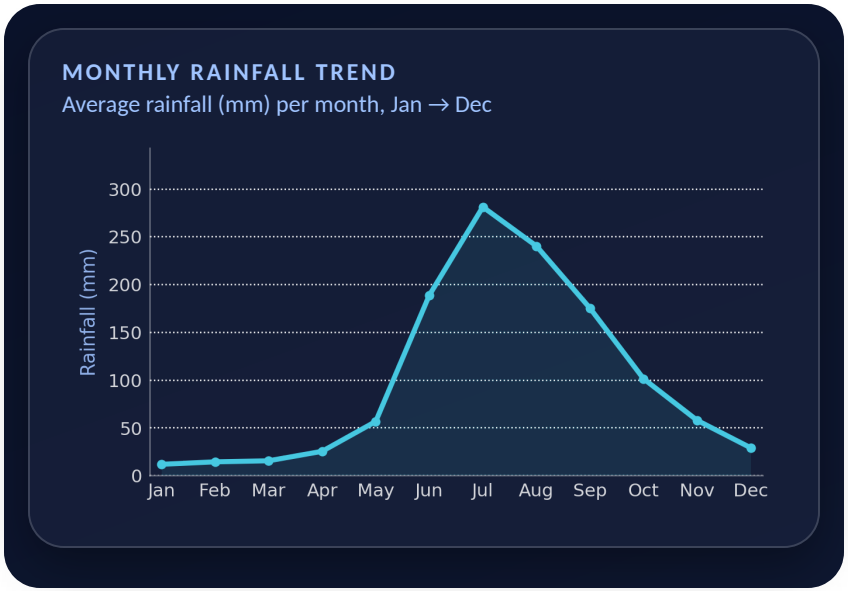
Dashboard Interface



Pluvia Analytics dashboard – header, filters, forecast, KPI cards and live charts (All Cities · All Months)

Data Visualization





❖ **Matplotlib** (headless Agg backend) renders transparent PNGs in the app palette, re-drawn on every filter or data change

❖ **Key insight:** rainfall is strongly seasonal – the monsoon (Jun–Sep) peaks in July at 281 mm average; Mumbai is wettest (1,920 mm/yr) and Delhi driest (671 mm/yr)

❖ The selected city is highlighted in sky blue while the others dim, so comparisons always work

❖ A third chart plots the **last 7 observed days** per city (rainfall or max temperature)

Forecast & Live Weather

Next-month forecast

One straight line across Jan–Dec would mislead (seasonality), so the app:

1. Fits a **local linear regression** over the trailing 4 months
2. Extrapolates one month ahead
3. Blends it: `0.4 × trend + 0.6 × seasonal`

Alert badge by projected rainfall

Heavy Monsoon Expected	≥ 200 mm
Elevated Rainfall	≥ 100 mm
Moderate Rainfall	40 – 100 mm
Dry Season Phase	< 40 mm

Live weather · Open-Meteo

- ❖ **Geocoding** resolves any city name (Indian match preferred)
- ❖ **7-day forecast cards**: condition, max/min °C, rain mm, rain probability
- ❖ **Last 7 days observed** chart for all cities (rainfall or temperature)
- ❖ **Sync Live Data** adds the latest complete month for the selected city
- ❖ Offline-safe: on failure the app keeps bundled / manual data and shows a message

Forecast is an illustrative estimate – not a meteorological prediction.

Export & Summary Report

Export Report dialog

CSV — **Filtered dataset**
Records for the current City / Month scope

PDF — **Rainfall summary**
Single-page report with bold KPI labels

TXT — **Rainfall summary**
Plain-text KPI summary

The **Summary** button opens the same report in-app, so the output always matches the active filters.

Sample summary (all cities, all months)

```
PLUVIA ANALYTICS — RAINFALL SUMMARY
Scope: All Cities · All Months
Records: 72
Total rainfall: 7,173.2 mm
Average rainfall: 99.6 mm
Maximum rainfall: 636.2 mm (Mumbai, Jul)
Minimum rainfall: 0.3 mm (Mumbai, Apr)

Forecast (Nov): 53.9 mm
Insight: Moderate Rainfall
Linear trend over 4 trailing month(s) + seasonal
norm · illustrative estimate, not a weather
forecast
```

Conclusion & Future Scope

Conclusion

✔ Total, average, maximum and minimum rainfall calculated automatically

✔ Summary reports and interactive visualizations delivered

✔ Records added manually, imported from CSV or synced live

✔ Phase 1 console logic evolved into a working, responsive dashboard

✔ Core functions are pure and composable, making them easy to test

Future Scope

❖ Persistent storage (SQLite) – records are currently held in memory per session

❖ More cities and a year-wise rainfall history

❖ ML-based rainfall prediction to replace the heuristic forecast

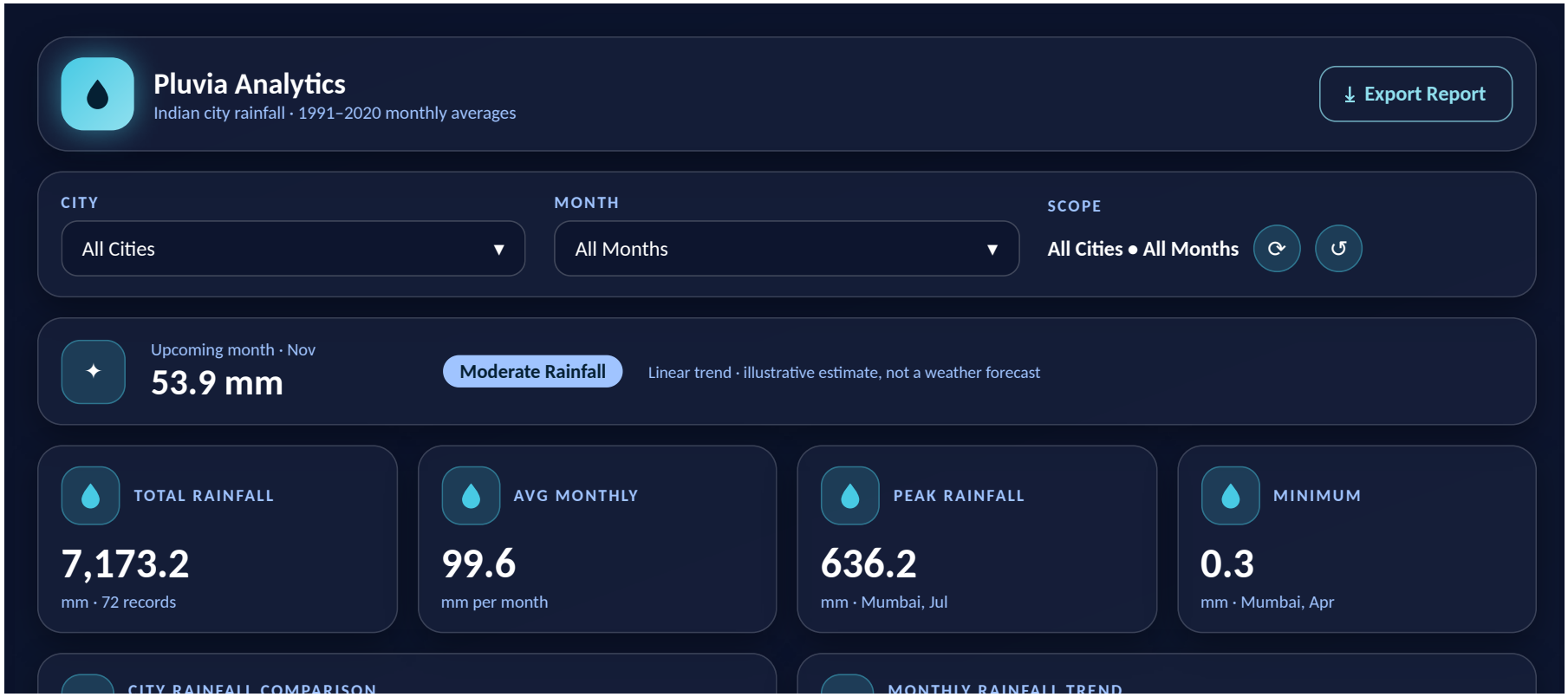
❖ Packaged desktop / mobile builds and automated unit tests

Pluvia Analytics
RAINFALL DATA INTELLIGENCE

GitHub

GitHub Account – Swanand-GH

Repository - rainfall-data-analyzer(Python)



Thank You